

BC3413/3407
(2024 S2)

Lesson 1 Recap - See speaker notes

```
def task1_summing_numbers(numbers):
    total = 0 # initialize a total variable
    for index, data in enumerate(numbers):# for loop to loop through the
index and number
        if index % 2 == 0:# if the current index is even index 0, 2, 4...
            total += data    # add the number to total
            print("..adding ", data)
    print("The total is ", total)
    return total# return the total
```

```
num_list = [21, 2, 56, 70, 80, 40, 21, 18]
task1_summing_numbers(num_list)
```

```
def task3():
    # initialize the string_text variable
    # split the string_text into list and keep
in a list var
    # initialize vowels list var
    # initialize consonant list var
    # loop through the string_text
        # if current word first letter in
aeiouAEIOU
            # append current word to vowels list
        # else
            # append to consonant list
```

```
def task2_keyword_search():
    string_list = []
    user_input = input("Enter a sentence:")# prompt user
for sentence input
    keyword = "fake" # initilize a string keyword
containing "fake"
    filtered_list = [] # initilize another filtered_list
starts with empty list
    while len(user_input) > 0:
        string_list.append(user_input)# append the sentence
to string_list
        if keyword in user_input:# if keyword in the
sentence
            filtered_list.append(user_input)# append to
filtered list
        user_input = input("Enter a sentence:")# prompt for
user input again
    print("The string containing keywords:", filtered_list)
```

```
task2_keyword_search()
```

```
# Task 4
test_string = "Python can mix and match."
text_list = test_string.split() # split by spacing
text_length = []
for each in text_list:
    text_length.append((each, len(each)))
print("The word and length pairs: ", text_length)
```

Lesson 2 - Demo

```
def split_text(sentence):
    sentence = sentence.replace(",","")
    sentence = sentence.replace(".", "")
    vowels = []
    consonants = []
    for word in sentence.split():
        word = word.lower()
        if word[0] in "aeiou":
            vowels.append(word)
        else:
            consonants.append(word)
    return vowels, consonants
```

```
sentence = input("Enter a sentence:")
name = input("Your name pls:")
vowels, consonants = split_text(sentence)
files_data = {
    f"{name}_vowels.txt": vowels,
    f"{name}_consonant.txt": consonants
}
for file, data in files_data.items():
    with open(file, "w") as fp:
        print("\n".join(data), file=fp)
```

```
import requests, webbrowser
```

```
url = "https://pokeapi.co/api/v2/pokemon/"
response = requests.get(url)
poke_data = response.json()
# check the data type of poke_data
print(poke_data['results']) # retrieve the name and url of pokes from
                             'results' key.
poke_names_url = {} # create a dictionary to keep the key and url.
for each in poke_data['results']:
    poke_names_url[each['name']] = each['url']
```

```
while True: # while loop to run the menu continuously
    menu = "Welcome to the world of Poke!\n" # Welcome message
    count = 1
    for name in poke_names_url.keys(): # for loop to loop through the names of
poke
        menu += f"{count:3}. {name:13}" # "{:10}".format(var)
        if count % 5 == 0:
            menu += "\n"
            count += 1
    print(menu) #, end="\n")
```

```
selection = input("Enter the name of poke:")
print(f"You have selection {selection}, you can see the details from
{poke_names_url[selection]}")
input("Press enter to open in browser...")
webbrowser.open(poke_names_url[selection])
if input("Continue? Y/N") == "N":
    break
```

```
# Task 3
import json
filename = "humidity.txt"
with open(filename, 'r') as file:
    with open("humidity_json.json", "w") as j_file:
        data = file.readlines()
        humidity = {}
        for index, line in enumerate(data):
            period, percent = line.strip().split()
            if index == 0:
                humidity['name'] = "Humidity Data"
                humidity['meta'] = {
                    'Period': period,
                    'Percent': percent
                }
                humidity['data'] = []
            else:
                humidity['data'].append({
                    'period': period,
                    'percent': percent
                })
        json.dump(humidity, j_file, indent=4)
```

Week 3 - Demo

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

#
# with open("Week3_page1.html", "r") as r:
#     soup = BeautifulSoup(r.read(), 'html.parser')
#     print(soup.find("p"))
#     print(soup.find("p"))

url = "https://www.channelnewsasia.com/"
r = requests.get(url)
soup = BeautifulSoup(r.content, 'html.parser')
# Utilize find_all to get the relevant tags from html file.
# <type class="class_name"> From the name of tags, fill in the
# relevant tags information below.
"""
<h6 class="h6 list-object__heading">
<a class="h6__link list-object__heading-link" href="/singapore/pm-
lawrence-wong-deliver-singapore-2025-budget-statement-parliament-feb-
18-4912286">
    Singapore's 2025 Budget statement to be delivered on Feb 18 at
3.30pm
</a>
</h6>
"""
# Signature: find_all(name, attrs, recursive, text, limit, **kwargs)
data= soup.find_all("h6", "h6 list-object__heading")
#print(data)
# print(data) # Take a look at the output before proceed.
count = 1
for each in data: # loop through all data
    # Get text from tag
    each_title = each.text # Get the text information from each title.
    print(f"{count}.{each_title.strip()}")# print out the title one by
one
    count += 1
```


Web Scraping - Task 2

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

url = "https://www.courts.com.sg/computing-mobile/tablets"
r = requests.get(url)
soup = BeautifulSoup(r.content, 'html.parser')

#%%%

# price = soup.find_all('span')
df = pd.DataFrame({})

# Tags for product title: <h3 class="product name product-item-name" style="height: 129px;">
product_title = soup.find_all("h3", "product name product-item-name")
print(product_title)
product_titles = []
for title in product_title:
    # Get info from tag
    each_title = title.text.strip()
    product_titles.append(each_title)

#%%%

product_price_divs = soup.find_all("div", "price-box price-final_price")
product_prices = []

for div in product_price_divs: # loop through all product titles
    each_price = div.find("span", "price-wrapper")
    print(each_price.get("data-price-amount"))
    product_prices.append(each_price) # append each title to the list.
product_prices
```

Web Scraping - Task 2 (cont')

```
def get_soup(url):
    r = requests.get(url)
    soup = BeautifulSoup(r.content, 'html.parser')
    return soup
def get_products(soup, tagname, classname, result):
    for each in soup.find_all(tagname, classname):
        result.append(each.text.strip())
    return result
def get_prices(soup, tagname, classname, subtag, subclass, attributename, result):
    for each in soup.find_all(tagname, classname):
        tag = each.find(subtag, subclass)
        result.append(float(tag.get(attributename)))
    return result

web_url = "https://www.courts.com.sg/computing-mobile/laptops/windows-laptop"
soup = get_soup(web_url)
tagname = "h3"
classname = "product name product-item-name"
result = []
result= get_products(soup, tagname, classname, result)

tagname = "div"
classname = "price-box price-final_price"
subtag = "span"
subclass = "price-wrapper"
attributename = "data-price-amount"
result_prices = []
result_prices = get_prices(soup, tagname, classname, subtag, subclass, attributename, result_prices)
print(len(result), result)
print(len(result_prices), result_prices)
###

product_titles = []
prices = []
for page in range(2,4):
    url = " https://www.courts.com.sg/computing-mobile/tablets?p= " + str(page)
    soup = get_soup_from_url(url)
    product_titles = get_titles(soup, product_titles)
    prices = get_prices(soup, prices)

df = pd.DataFrame({})
df['product_titles'] = product_titles
df['each_price'] = prices
###

df.to_csv("tablet_prices.csv")
```

Week 5- OOP Demo

```
class Person:

    def __init__(self, name, email=""):

        self.name = name

        self.email = email


    def introduction(self):

        print(f"I'm {self.name}, email:{self.email}")


    def __str__(self):

        return f"{self.name} string representation."


class Student(Person):

    def __init__(self, name="", matric="", email=""):

        super().__init__(name, email)

        self.matric = matric


    def __str__(self):

        return f"I'm a full time student {self.name}, matric={self.matric}"

#####

john = Person("John")

john.introduction()

mary = Person(name="Mary", email="mary@gmail.com")

mary.introduction()


print(type(mary), mary)

print(john)


jimmy = Student("Jimmy", matric="U201729A")

print(jimmy)
```

Week 5 - OOP Cont (Task 2)

```
class News:

    def __init__(self, news):

        self.news_headline = news[0]

        self.news_article = news[1]

        self.news_category = news[2]

        self.full_text = news[3]

        self.clean_text = news[4]

        self.words = len(self.full_text)

    def __str__(self):

        return f"<{self.words} words> in {self.news_category}: {self.news_headline} "
```



```
class FinancialNews(News):

    def __init__(self, news):

        # Symbol,news_headline,news_article,news_category,full_text,clean_text

        super().__init__(news[1:])

        self.symbol = news[0]

    def __str__(self):

        return f"<{self.symbol}, <{self.words} words> in {self.news_category}: {self.news_headline} "
```



```
import csv

news_instances = []

with open("news.csv", "r", encoding="utf-8") as f:

    csv_pointer = csv.reader(f)

    line_no = 0

    for line in csv_pointer:

        if line_no != 0:

            news_obj = News(line)

            news_instances.append(news_obj)

        line_no += 1

with open("finance_news.csv", "r", encoding="utf-8") as f:

    csv_pointer = csv.reader(f)

    line_no = 0

    for line in csv_pointer:

        if line_no != 0:

            news_obj = FinancialNews(line)

            news_instances.append(news_obj)

        line_no += 1

for i in range(19):

    print(news_instances[i])

for i in range(19):

    print(news_instances[-i])
```


Week 6 - OS & DB

```
import random

print(random.randrange(3000, 6000))

# Discuss at 11:15

salaries = [random.randrange(3000, 6000) for salary in range(50)]

salaries_high = [salary for salary in salaries if salary > 4500]

salaries_high_format = [f"${salary:,.0f}" for salary in salaries if salary > 4500]

print("Salaries:", salaries)

print("Selected Salaries: ", salaries_high_format)
```

```
import sqlite3

# establish connection

conn = sqlite3.connect("salaries.db")

# get cursor

cursor = conn.cursor()

# SQL

SQL_stmt = "SELECT * FROM salaryTable;"

cursor.execute(SQL_stmt)

for each in cursor.fetchall():

    print(each[:2])
```

Group 7

```
df = pd.read_csv("employee.csv")

# records, cols
print("records, cols:", df.shape)
# duplicates
print("duplicates:", df[df.duplicated()].shape[0])
# same employee No.
print("same employee no.:", df[df['EmployeeNumber'].duplicated()].shape[0])
# null records
print("null records:", df[df.isna()].shape[0])
```

records, cols: (8687, 37)

duplicates: 14

same employee no.: 31

null records: 8687

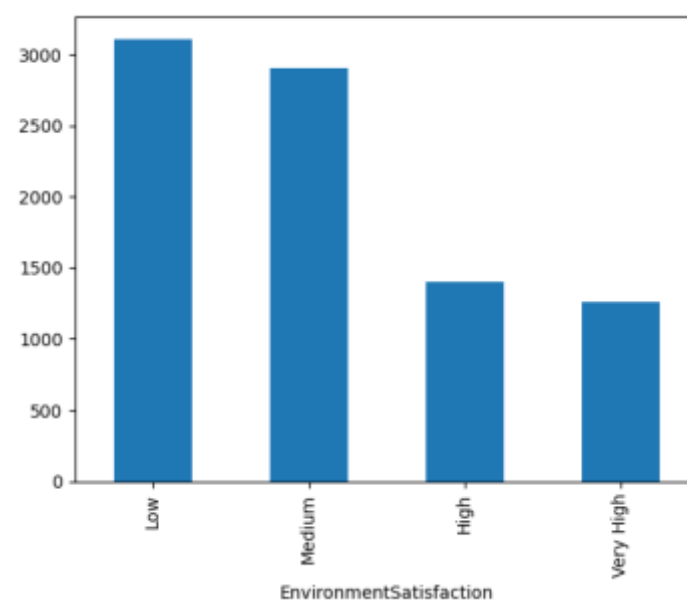
```
conversion_rate = 1.35
df['DailyRate_SGD'] = list(map(lambda x:x*conversion_rate, df['DailyRate']))

df['WorkLifeBalance']=df["WorkLifeBalance"].map({1:"Bad", 2:"Good", 3:"Better", 4:"Best"})
```

```
df[df["MonthlyIncome"] == "Married"]["MonthlyIncome"]
```

Monthly Income contains Married

Low Environment satisfaction



Group 3

Task 2

Q1 - 8687

Q2 - 37

Q3 - 14

Q4 - 31

Q5 - 139

Workings:

`df.shape` #No. of records, no. of columns

`df.duplicated().sum()` #no. of duplicated records

`df.EmployeeNumber.duplicated().sum()` #no. of duplicated Employee Number

`df.isna().sum().sum()` #no. of missing records

Group 6

- Q1
 - 8687 records
- Q2
 - 37 columns
- Q3
 - 14 duplicated records
- Q4
 - 31 records with same EmployeeNumber
- Q5
 - `employee.isna().sum()`

Group 4

```
print(len(df))
```

```
#8687
```

```
print(df.shape[1])
```

```
#37
```

```
print(df.duplicated().sum())
```

```
#14
```

```
print(df.EmployeeNumber.duplicated().sum())
```

```
#31
```

```
print(df.isna().sum())
```

Group 4

8687 records

37 columns

14 duplicated records

31 duplicated records with same EmployeeNumber

139 null records

```
num_records = df1.shape[0]
print(num_records)
```

```
num_columns = df1.shape[1]
print(num_columns)
```

```
num_duplicated_records = df1.duplicated().sum()
print(num_duplicated_records)
```

```
num_duplicated_employee_numbers = df1.duplicated(subset=["EmployeeNumber"]).sum()
print(num_duplicated_employee_numbers)
```

```
num_null_records = df1.isnull().sum().sum()
print(num_null_records)
```

Group 8

```
import pandas as pd

df = pd.read_csv("employee.csv")

df

#

print("Number of rows:", len(df))
```

8687

```
print("Number of columns:", len(df.columns))
```

37

```
duplicate_rows = df[df.duplicated()]

print(duplicate_rows)

print("Number of duplicate rows:", len(duplicate_rows))
```

14

```
duplicate_id = df["EmployeeNumber"].duplicated()

print(duplicate_id)

print("Number of duplicate EmployeeNumbers:", duplicate_id.sum())
```

31

-
-
-

Group 2

```
import pandas as pd
```

Task 2:

```
df = pd.read_csv("employee.txt")
```

```
df
```

```
# 1. Number of records
```

```
num_records = df.shape[0]
```

```
# 8687 records, 37 columns
```

```
# 2. Number of columns
```

```
num_columns = df.shape[1 ]
```

```
#37
```

```
# 3. Number of fully duplicate
```

```
num_exact_duplicates = df.duplicated().sum()
```

```
# 14
```

```
# 4. Number of rows that share the same EmployeeNumber
```

```
num_dup_empnumber = df["EmployeeNumber"].duplicated().sum()
```

```
#31
```

```
# 5. Missing values
```

```
missing_by_column = df.isna().sum()    # Missing values per column
```

```
total_missing = missing_by_column.sum() # Total missing values
```

```
# Number of null records: 139
```


Group 5

1. How many records are there in this data file?

Ans: 8687

2. How many columns are there in this data file?

Ans: 37

3. Any duplicated records (records with all columns same values)?
How many of them?

Ans: 14

4. Any records with the same EmployeeNumber? How many of them?

Ans: 31

5. Are there any null records? If so, how many of them?

Ans: 139



Week 7 - Employee.csv

See speaker notes

Web Programming - Flask

New Project

Pure Python

Django

Flask

Google App Engine

Pyramid

Web2Py

Scientific

Angular CLI

AngularJS

Bootstrap

HTML5 Boilerplate

React

React Native

Location: \OneDrive - Nanyang Technological University\My Subjects\BC3407 BC3413\Code 2025\FlaskProj

Python Interpreter: New Virtualenv environment

New environment using

Virtualenv

Location: anyang Technological University\My Subjects\BC3407 BC3413\Code 2025\FlaskProj

Base interpreter: C:\Users\yokye\AppData\Local\Programs\Python\Python310\python.exe

☐ Inherit global site-packages

☐ Make available to all projects

☐ Previously configured interpreter

Interpreter: <No interpreter>

More Settings

CreateCancel